

Specs are defined by calling the global Jasmine function `it()`, which, like `describe()`, takes a string and a function. The string is the title of the spec and the function is the spec, or test. A spec contains one or more expectations that test the state of the code. An expectation in Jasmine is an assertion that is either true or false. A spec with all true expectations is a *passing* spec. A spec with one or more false expectations is a *failing* spec.

Here is a sample format of `spec.js`:

```
describe('Protractor Demo App', function() {
  it('title of your choice', function() {
    browser.get('http://juliemr.github.io/protractor-demo/');
    expect(browser.getTitle()).toEqual('title of the
    webpage');
  });
});
```

Using Protractor for non-AngularJS applications

Protractor is majorly used for creating tests for Angular applications. However, support is available for non-Angular pages as well. To use Protractor for non-AngularJS applications:

1. Use `browser.driver` instead of `driver`
2. Use `browser.driver.ignoreSynchronization = true`
3. Use `browser.waitForAngularEnabled(true)`

Why should we use a different method for non-AngularJS applications? In Angular applications, Protractor starts when the whole Angular content gets loaded on the Web page. The reason we use the different methods is that Protractor should not wait for the Angular components

to load completely. However, since our pages are non-Angular, Protractor keeps waiting for Angular to load till the test fails with a timeout. So, we need to explicitly tell Protractor not to wait for Angular.

By using `browser.driver`, we tell it that there is no Angular content to be loaded. So the test should run. `browser.driver.ignoreSynchronization = true` forces Protractor not to wait for Angular to load and finish its tasks. `browser.waitForAngularEnabled(true)` is used when you want to load an Angular application after loading a non-Angular application. After executing this line, it will wait for Angular content to be loaded and start execution.

Here is a simple snapshot of what a non-Angular test case will look like:

```
browser.ignoreSynchronization=true;
browser.driver.get('https://yourNonAngularApplicationLink');
element(by.id('username')).sendKeys('ab');
element(by.id('password')).sendKeys('1234');
element(by.id('clickme')).click();
browser.waitForAngularEnabled(true);
browser.get('https://yourAngularApplicationLink');
```



By: Astha Srivastava

The author is a software developer. Her areas of expertise are C, C++, C#, Java, JavaScript, HTML and ASP.NET. She has recently started working on the basics of artificial intelligence. She can be reached at asthasri25@gmail.com.

OSFY Magazine Attractions During 2018-19

MONTH	THEME
March 2018	Databases management and Optimisation
April 2018	Docker and containers
May 2018	Open Source Programming (Languages and tools)
June 2018	Open Source Firewall and Network security and Monitoring
July 2018	Mobile App Development and Optimisation
August 2018	AI, Deep learning and Machine Learning
September 2018	Open Source and IoT
October 2018	BigData, Hadoop, PaaS, SaaS, IaaS and Cloud
November 2018	Open Source on Windows
December 2018	Web Development
January 2019	Data Security, Storage and Backup
February 2019	Best in the world of Open Source (Tools and Services)